1、Flink核心原理与高频面试

由 涤生老师-健哥创建, 最后修改于七月 31, 2026

- 0. 对于Flink相关面试如何准备的建议
- 1、什么是Flink？（必背）
- 2、Flink常见的使用场景（了解）
- 3、Flink相比SparkStreaming有什么区别（了解）
- 4、Flink常见的组件栈有哪些（了解）
- 5、Flink运行时的架构组件有哪些？各自有什么作用？（理解背诵）
- 6、Flink的任务提交流程（理解背诵）
- 7、任务调度原理（理解背诵）
- 8、如何理解Flink中的Slot（必背）
- 9、Flink的核心数据集DataStream的组成（理解背诵）
- 10、Flink中是否也有Spark中的宽窄依赖的概念（了解）
- 11、Flink中的四张图（了解）
- 12、你知道Flink中的任务链吗？（了解）
- 13、Flink中常见的任务状态及如何转换（了解）
- 14、Flink核心的四大基石是什么？（背诵）
- 15、Flink的并行度是什么？（理解背诵）
  - 15.1 任务槽和并行度的关系？（理解背诵）
  - 15.2、流式计算我们一般希望并行处理、任务之间相互隔离，为什么Flink又允许共享 slot 呢？谈谈你的理解？（理解）
- 16 . 谈谈Flink 回撤机制的理解

0. 对于Flink相关面试如何准备的建议

讲讲我的经验, 针对实时的真实企业面试, 目前已经不是一个单纯的知识点面试, 而是结合具体问题出发, 考察你的实时架构设计能力、数据质量保障, 业务理解深度以及生产环境的问题排查经验。

通常面试官会关注项目背景、技术选型、系统设计、遇到的问题及解决方案，以及取得的成果。需要将这些问题分解成几个部分，并给出每个部分的关键点。

项目介绍：

- 举例说明：“在我们XX项目中，选择Flink是因为需要处理每分钟10万+订单的实时规则计算，且要求保障Exactly-Once”

故障案例：

- “曾遇到Checkpoint超时，发现是因为背压导致Checkpoint，分析背压原因主要是资源分配问题，通过调整资源分配并开启增量Checkpoint后恢复”

莫哥的面试准备建议

- **准备一个你认为的比较问题案例**：提前复盘一个线上故障或性能瓶颈，按照“现象 -> 排查过程 -> 根因分析 -> 解决方案 -> 量化效果 -> 复盘改进”的结构准备，这比背八股更有说服力。
- **理解业务本质**：不要只谈技术，要思考技术如何服务于业务目标。例如，为什么这个场景要用实时？带来了什么业务价值？
- **关注实时技术发展趋势**：了解湖仓一体、流批一体等前沿概念，并能说出自己的技术选型和理解。

1、什么是Flink？（必背）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 非常基础, 尤其是对于了解flink的小白来讲,会让谈谈对flink的理解;** 2. **update 2026.07**

Apache Flink是一款开源流处理框架，由Apache软件基金会进行维护和开发，**专为实时数据处理、批处理以及流批一体的大规模数据处理场景而设计**。Flink的核心理念是提供了一种统一的数据处理模型，使得无论是实时流处理还是批量处理，都可以在同一个系统中以一致的方式高效执行。

Flink的设计基于数据流编程模型，其核心特性包括事件时间处理（Event Time Processing）、精确一次状态一致性（Exactly-once State Consistency）和故障恢复机制，这些都确保了在高并发、大数据量下对数据进行低延迟、高吞吐且准确无误的处理。

1. **首先，Flink支持事件时间处理，**这意味着它能够根据数据本身携带的时间戳进行计算，而非系统的处理时间，从而更好地处理乱序事件和窗口计算问题，尤其适合于实时业务监控、预警等场景。
2. **其次，Flink提供了精确一次的状态一致性保证，**即使在发生故障或重启的情况下，也能确保每条记录只被处理一次，并且中间结果的状态保持正确。这对于需要维护用户会话、统计累积指标等有状态计算任务至关重要。
3. **再者，Flink具备高度的可扩展性和容错性，**其分布式运行环境能很好地适应大规模集群部署，通过checkpoint机制实现快速故障恢复，保障服务的高可用性。
4. **此外，Flink支持丰富的连接器接口，**可以无缝对接各种数据源和数据接收系统，如Kafka、HDFS、MySQL、Elasticsearch等，方便企业构建端到端的数据处理管道。Flink还具有动态资源调整、SQL & Table API等多种易用性功能，使开发者无需深入了解底层细节，即可快速构建复杂的数据处理应用。

总的来说，Apache Flink凭借其强大的实时处理能力、灵活的编程模型、严格的精确一次语义以及广泛的数据源适配性，在大数据处理领域占据了重要地位，已在全球范围内被诸多互联网公司、金融机构以及其他行业的企业广泛应用，有效推动了实时数据分析与决策的发展。

2、Flink常见的使用场景（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 一般结合项目来问,面试几乎不会被问到,不是重点;** 2. **update 2026.07**

a、事件驱动Event-driven Applications

典型案例：

i、欺诈检测(Fraud detection)

ii、异常检测(Anomaly detection)

iii、基于规则的告警(Rule-based alerting)

iv、业务流程监控(Business process monitoring)

v、Web应用程序(社交网络)

b、数据分析Data Analytics Applications

典型实例：

i、电信网络质量监控

ii、移动应用中的产品更新及实验评估分析

iii、消费者技术中的实时数据即席分析

iv、大规模图分析

c、数据管道Data Pipeline Applications

典型实例：

i、电子商务中的持续 ETL(实时数仓)

ii、电子商务中的实时查询索引构建(搜索引擎推荐)

3、Flink相比SparkStreaming有什么区别（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 面试偶尔会被问到, 频率不高, 高级和资深同学可以了解;** 2. **update 2026.07**

1、架构模型

Spark Streaming 在运行时的主要角色包括：Master、Worker、Driver、Executor

Flink 在运行时主要包含：Jobmanager、Taskmanager、 Client 、Slot。

2、任务调度

Spark Streaming 连续不断的生成微小的数据批次，构建有向无环图 DAG，Spark Streaming 会依次创建 DStreamGraph、JobGenerator、JobScheduler。Flink 根据用户提交的代码生成 StreamGraph，经过优化生成 JobGraph，然后提交给 JobManager 进行处理，JobManager 会根据 JobGraph 生成 ExecutionGraph，ExecutionGraph 是 Flink 调度最核心的数据结构，JobManager 根据ExecutionGraph 对 Job 进行调度。

3、时间机制

Spark Streaming 支持的时间机制有限，只支持处理时间。

Flink 支持了流处理程序在时间上的三个定义：处理时间、事件时间、摄入时间。同时也支持watermark 机制来处理滞后数据。

4、容错机制

对于 Spark Streaming 任务，我们可以设置 checkpoint，然后假如发生故障并重启，我们可以从上次checkpoint 之处恢复，但是这个行为只能使得数据不丢失，可能会重复处理，不能做到恰一次处理语义，如果我们想要实现Exactly-once，需要自己实现。

Flink 内部提供了Exactly-once实现，使用起来较为方便

5、其他角度

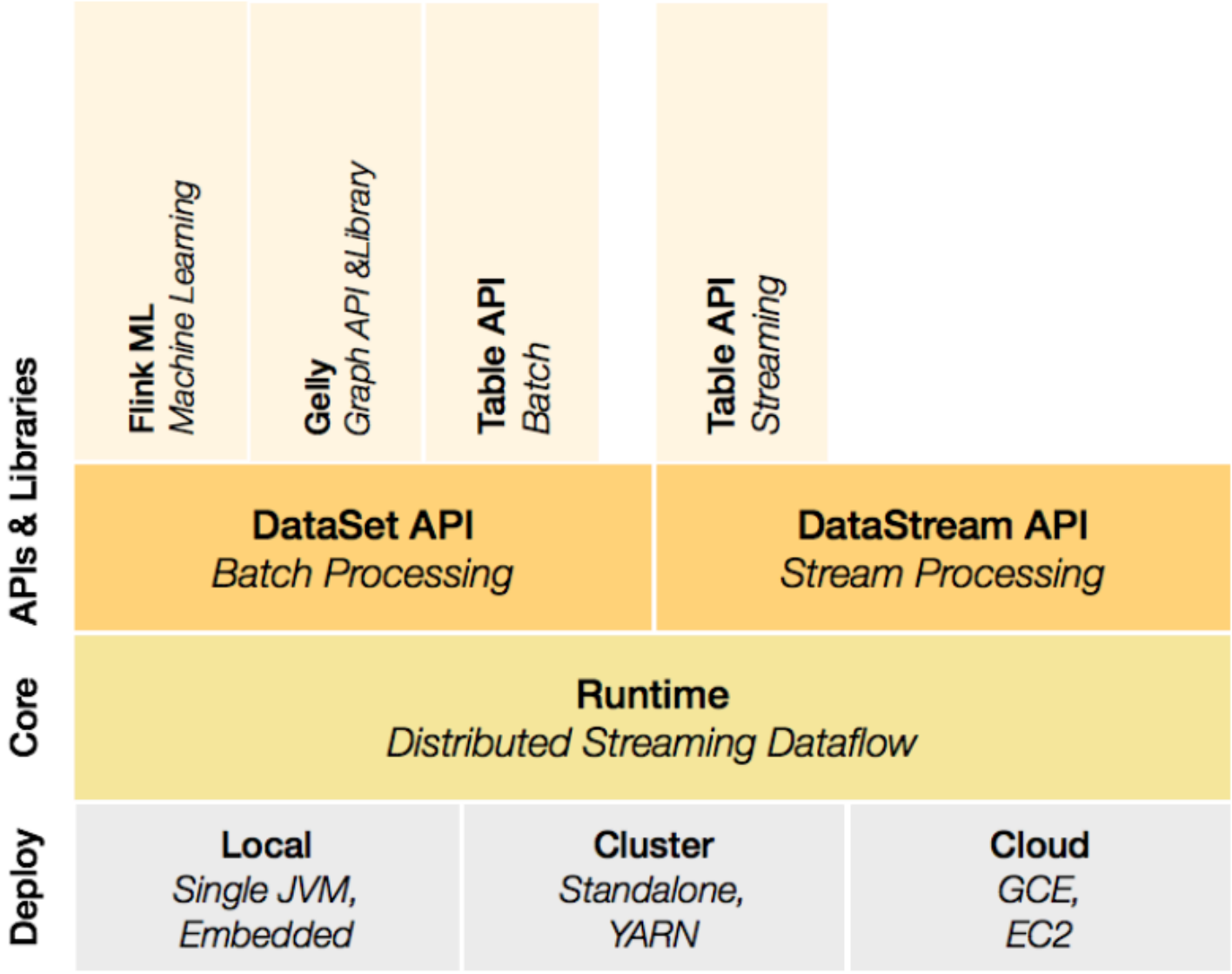
state，window等等

参考建议：这个是一个非常灵活的面试题，也是一个比较好的面试题，一方面可以考察出来你对Spark Streaming和Flink的了解的程度。一方面可以考察出来你对实时计算理解的程度。这个题的回答答案不唯一，但是回答的适合思路一定要清晰。千万别柔在一起回答

4、Flink常见的组件栈有哪些（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 面试偶尔会被问到, 了解既可;** 2. **update 2026.07**

Flink是一个分层架构的系统，每一层所包含的组件都提供了特定的抽象，用来服务于上层组件。



Deployment层：该层主要涉及了Flink的部署模式，Flink支持多种部署模式：本地、集群（Standalone/YARN）、云（GCE/EC2）。

Runtime层：Runtime层提供了支持Flink计算的全部核心实现，比如：支持分布式Stream处理、JobGraph到ExecutionGraph的映射、调度等等，为上层API层提供基础服务。

API层：主要实现了面向无界Stream的流处理和面向Batch的批处理API，其中面向流处理对应DataStream API，面向批处理对应DataSet API。

Libraries层：该层也可以称为Flink应用框架层，根据API层的划分，在API层之上构建的满足特定应用的实现计算框架，也分别对应于面向流处理和面向批处理两类。面向流处理支持：CEP（复杂事件处理）、基于SQL-like的操作（基于Table的关系操作）；面向批处理支持： FlinkML（机器学习库）、Gelly（图处理）。

5、Flink运行时的架构组件有哪些？各自有什么作用？（理解背诵）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 面试经常会被问到, 非常基础非常核心;** 2. **update 2026.07**

Flink运行时架构主要包括四个不同的组件，它们会在运行流处理应用程序时协同工作：作业管理器（JobManager）、资源管理器（ResourceManager）、任务管理器（TaskManager），以及分发器（Dispatcher）。因为Flink是用Java和Scala实现的，所以所有组件都会运行在Java虚拟机上。每个组件的职责如下：

1）作业管理器（JobManager）

控制一个应用程序执行的主进程，也就是说，**每个应用都应该被唯一的 JobManager 所控制执行**。**JobManager会先接收到要执行的应用程序，这个应用程序会包括：作业图（JobGraph）、逻辑数据流图（logical dataflow graph）和打包了所有的类、库和其它资源的JAR包。**JobManager会把JobGraph转换成一个物理层面的数据流图，这个图被叫做“执行图”（ExecutionGraph），包含了所有可以并发执行的任务。**JobManager会向资源管理器（ResourceManager）请求执行任务必要的资源，也就是任务管理器（TaskManager）上的插槽（slot）**。一旦它获取到了足够的资源，就会将执行图分发到真正运行它们的TaskManager上。而**在运行过程中，JobManager会负责所有需要中央协调的操作，包括任务调度，检查点管理，失败恢复等**。当然，对于集群HA模式，可以同时多个master进程，其中一个作为leader，其他作为standby。当leader失败时，会选出一个standby的master作为新的leader（通过zookeeper实现leader选举）。

总结：JobManager的功能主要有：

- 将 JobGraph 转换成 Execution Graph，最终将 Execution Graph 拿来运行

- Scheduler 组件负责 Task 的调度
- Checkpoint Coordinator 组件负责协调整个任务的 Checkpoint，包括 Checkpoint 的开始和完成
- 通过 Actor System 与 TaskManager 进行通信
- 其它的一些功能，例如 Recovery Metadata，用于进行故障恢复时，可以从 Metadata 里面读取数据

JobManager 进程由三个不同的组件组成：**ResourceManager、Dispatcher和JobMaster**

i、资源管理器（ResourceManager）

主要负责管理任务管理器（TaskManager）插槽（slot）的资源提供、回收、分配等。Flink为不同的环境和资源管理工具提供了不同资源管理器，比如YARN、Mesos、K8s，以及Standalone部署。当JobManager申请插槽资源时，ResourceManager会将有空闲插槽的TaskManager分配给JobManager。**如果ResourceManager没有足够的插槽来满足JobManager的请求，它还可以向资源提供平台发起会话，以提供启动TaskManager进程的容器。另外，ResourceManager还负责终止空闲的TaskManager，释放计算资源。在整个 Flink 集群中只有一个 ResourceManager。**

需要注意的是，这里的ResourceManager和Yarn中的ResourceManager不是一回事儿。

ii、分发器（Dispatcher）

Dispatcher 提供了一个 REST 接口，用来提交 Flink 应用程序执行，**并为每个提交的作业启动一个新的 JobMaster**。由于是REST接口，所以Dispatcher可以作为集群的一个**HTTP接入点**，这样就能够不受防火墙阻挡。前面我们说Flink Web UI是JobManager启动的，受限于当时对Flink的了解。现在我们**严格上的来说Flink Web UI是Dispatcher启动的，用来方便地展示和监控作业执行的信息。**

iii、JobMaster

JobMaster 负责管理单个JobGraph的执行。Flink 集群中可以同时运行多个作业，**每个作业都有自己的 JobMaster**。

2）任务管理器（TaskManager）

Flink中的工作进程，通常在Flink中会有多个TaskManager运行，每一个TaskManager都包含了一定数量的插槽（slots），TaskManger插槽是Flink中定义的处理资源单元。插槽的数量限制了TaskManager能够执行的任务数量。**启动之后，TaskManager会向资源管理器注册它的插槽**；收到资源管理器的指令后，TaskManager就会将一个或者多个插槽提供给JobManager调用。**JobManager就可以向插槽分配任务（tasks）来执行了。**在执行过程中，一个TaskManager可以跟其它运行同一应用程序的TaskManager交换数据。

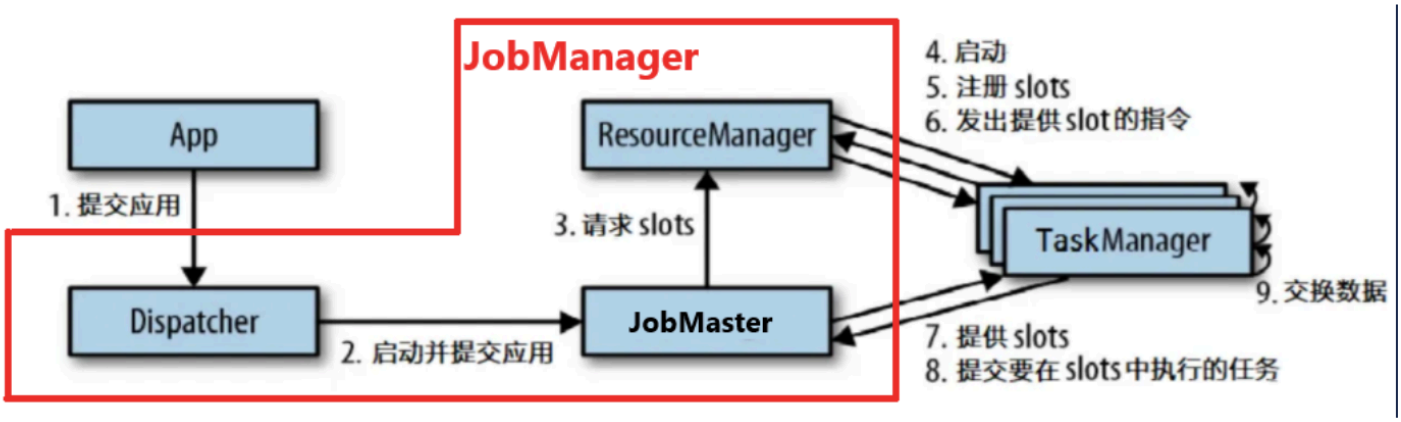
TaskManager 里面的主要组件有：

- Memory & I/O Manager，即内存 I/O 的管理
- Network Manager，用来对网络方面进行管理
- Actor system，用来负责网络的通信

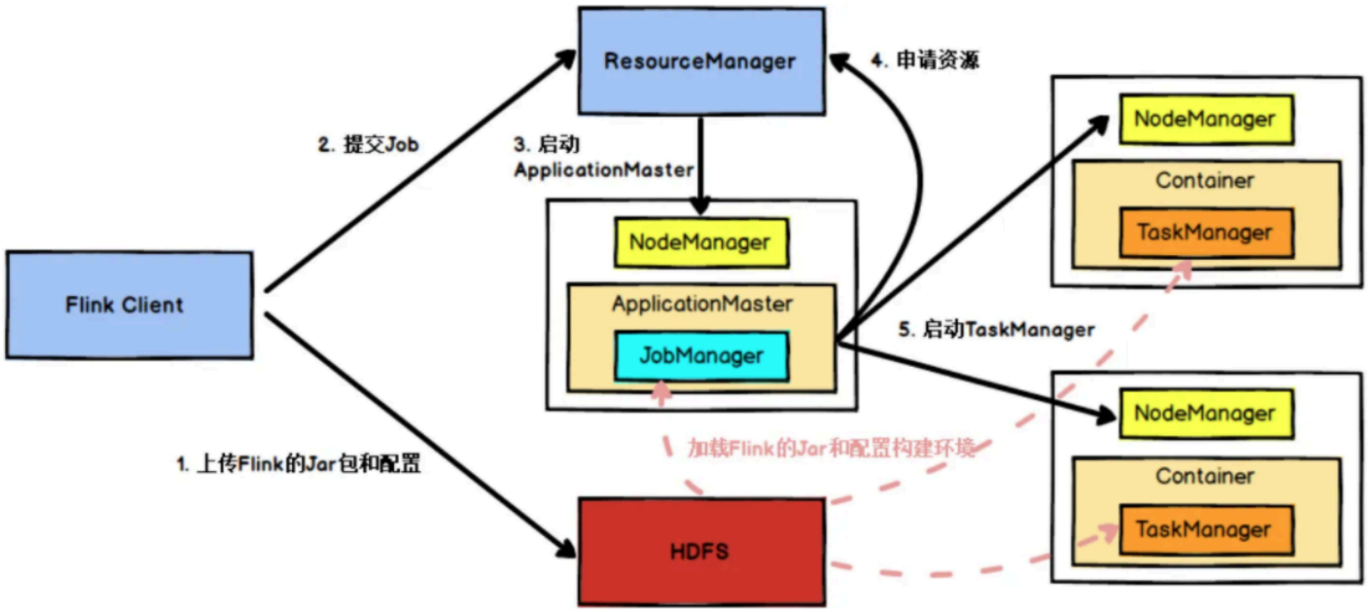
6、Flink的任务提交流程（理解背诵）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 对于刚做实时不久的同学, 面试偶尔会被问到, 非常基础非常核心;** 2. **update 2026.07**

当一个应用提交执行时，Flink的各个组件是如何交互协作的：



上图是从一个较为高层级的视角，来看应用中各组件的交互协作。如果部署的集群环境不同（例如YARN，Mesos，Kubernetes，standalone等），其中一些步骤可以被省略，或是有些组件会运行在同一个JVM进程中。具体地，如果我们将Flink集群部署到YARN上，那么就会有如下的提交流程：



i、Flink任务提交后，Client向HDFS上传Flink的Jar包和配置；

ii、之后向Yarn ResourceManager提交任务，**ResourceManager分配Container资源并通知对应的NodeManager启动ApplicationMaster；**

iii、ApplicationMaster启动后加载Flink的Jar包和配置构建环境，然后启动JobManager；

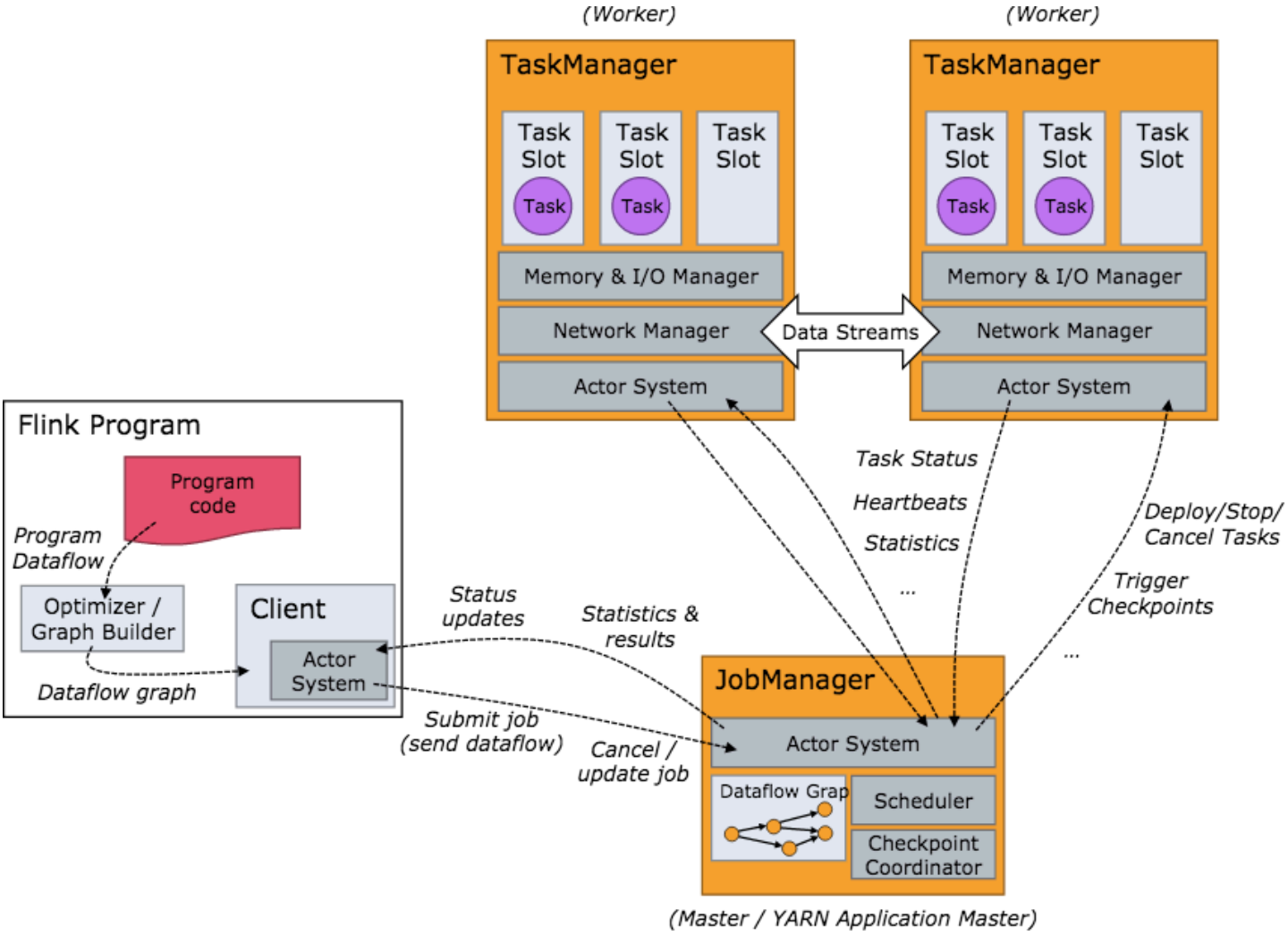
iv、之后ApplicationMaster向ResourceManager申请资源启动TaskManager，ResourceManager分配Container资源后，**由ApplicationMaster通知资源所在节点的NodeManager启动TaskManager；**

v、NodeManager加载Flink的Jar包和配置构建环境并启动TaskManager；

vi、TaskManager启动后向JobManager发送心跳包，并等待JobManager向其分配任务。

7、任务调度原理（理解背诵）

Update	Review	尖叫提醒
健哥	光哥	1. 尖叫提示: 对于刚做实时不久的同学,面试偶尔会被问到, 非常基础非常核心; 2. update 2026.07



客户端不是运行时和程序执行的一部分，但它用于准备并发送dataflow(JobGraph)给Master(JobManager)，然后，客户端断开连接或者维持连接以等待接收计算结果。

当 Flink 集群启动后，首先会启动一个 JobManger 和一个或多个的 TaskManager。由 Client 提交任务给 JobManager，JobManager 再调度任务到各个 TaskManager 去执行，然后 T**askManager 将心跳和统计信息汇报给 JobManager**。TaskManager 之间以流的形式进行数据的传输。上述三者均为独立的 JVM 进程。

8、如何理解Flink中的Slot（必背）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 面试经常会被问到, 非常基础非常核心;** 2. **update 2026.07**

Flink中每一个worker(TaskManager)都是一个**JVM进程，为了控制一个worker能接收多少个task，worker通过task slot来进行控制（一个worker至少有一个task slot）**。每个task slot表示TaskManager拥有资源的**一个固定大小的子集。Task的并行度是由运行在Task Manager中的task slots的数量决定。如果一个Task Manager有4个slots，那么JVM的内存将平均分配给每个task slot 25%的TaskManager的可用内存。**一个Task slot中可以运行1个或多个线程，同一个slot中的线程又可以共享相同的JVM。将资源 slot 化意味着来自不同job的task不会为了内存而竞争，而是每个task都拥有一定数量的内存储备。而**在同一个JVM进程中的task将共享TCP连接（基于多路复用）和心跳消息。它们也可能共享数据集和数据结构，因此这减少了每个task的负载。**

slot类似于spark中的task，只是worker节点上的一个线程，一般一个slot单独占用一个core

说明：一个taskmanager上的slot的个数由参数：taskmanager.numberOfTaskSlots决定，仅仅对内存做了隔离，并没有对cpu进行隔离。

9、Flink的核心数据集DataStream的组成（理解背诵）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 面试偶尔会被问到, 非常基础非常核心;** 2. **update 2026.07**

DataStream程序主要包含3部分：

1、StreamExecutionEnvironment初始化：该部分主要创建和初始化StreamExecutionEnvironment，提供通过DataStream API构建Flink作业需要的执行环境，包括设定ExecutionConfig、CheckpointConfig等配置信息以及StateBackend和TimeCharacteristic等变量
2、业务逻辑代码转换：env中提供了创建DataStream的方法，通过env.readTextFile()方法读取文本数据并构建DataStreamSource数据集，之后所有的

DataStream操作都会以DataStreamSource为头节点。同时DataStreamAPI 提供了各种转换操作，如map、reduce、keyBy等算子，用于构建完整的计算逻辑。

3、程序执行env.execute()：在execute()方法中基于DataStream之间的转换生成StreamGraph,并将StreamGraph转成成JobGraph,最终将JobGraph提交到集群

10、Flink中是否也有Spark中的宽窄依赖的概念（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 实时开发了解可以, 面试偶尔会被问到;** 2. **update 2026.07**

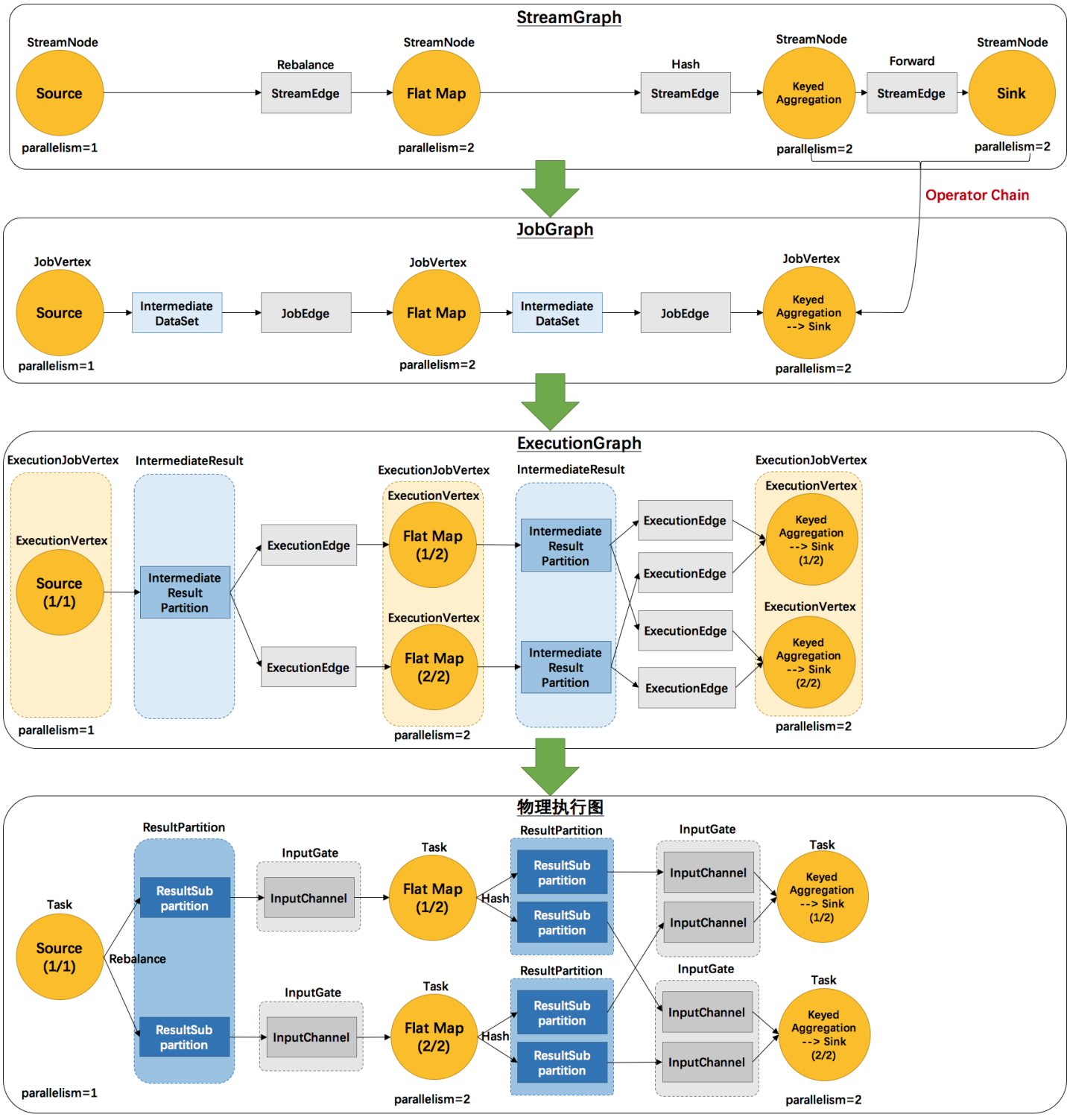
Stream 在算子之间传输数据的形式可以是 one-to-one(forwarding)模式，也可以是 redistributing模式，具体是哪一种形式，取决于算子的种类。

One-to-one：stream 维护着分区以及元素的顺序(比如在 source 和 map 之间)。那意味着 map 算子的子任务看到的元素的个数以及顺序跟 source 算子的子任务生产的元素的个数、顺序相同，map、fliter、flatMap 等算子都是 one-to-one 的对应关系。对于并行度一样的one-to-one操作，每个子任务可以合并程一个子任务。**类似于 spark 中的窄依赖**

Redistributing：stream 的分区会发生改变，每一个算子的子任务依据所选择的 transformation 发送数据到不同的目标任务。例如，keyBy() 基于 hashCode 重分区、broadcast 和 rebalance 会随机重新分区，这些算子都会引起 redistribute 过程，而 redistribute 过程就类似于Spark 中的 shuffle 过程，即**类似于 spark 中的宽依赖**

11、Flink中的四张图（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 实时开发必须掌握的, 但是面试比较少会被问到, 不算高频;** 2. **update 2026.07**



StreamGraph：根据用户通过DataStreamAPI编写的代码生成的最初的DAG图，用来表示程序的拓扑结构。这一步一般在**客户端**完成。

JobGraph：StreamGraph经过优化后生成的就是作业图（JobGraph），这是提交给JobManager 的数据结构，确定了当前作业中所有任务的划分。主要的优化为: 将多个符合条件的节点链接在一起合并成一个任务节点，形成算子链，这样可以减少数据交换的消耗。JobGraph一般也是在客户端生成的，在作业提交时传递给JobMaster。

ExecutionGraph：JobMaster收到JobGraph后，会根据它来生成执行图（ExecutionGraph）。ExecutionGraph是JobGraph的并行化版本，是调度层最核心的数据结构。生成位置：在jobmanager上生成。JobMaster收到JobGraph后，会根据它来生成执行图（ExecutionGraph）。

物理执行图：JobMaster 生成执行图后，会将它分发给 TaskManager；各个 TaskManager 会根据执行图部署任务，最终的物理执行过程也会形成一张“图”，一般就叫作物理图（Physical Graph）。这只是具体执行层面的图，并不是一个具体的数据结构。生成位置：在taskmanager上生成。

12、你知道Flink中的任务链吗？（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 实时开发必须了解, 面试偶尔会被问到;** 2. **update 2026.07**

Flink采用一种称为任务链（Operator Chain）的优化技术，相同并行度的 one to one 操作，Flink 把这样相连的算子链接在一起形成一个 task，原来的算子成为里面的一部分（subtask）。为了满足任务链的要求，必须将两个或多个算子设为相同的并行度，并通过本地转发（local forward）的形式

进行链接。在一个 DAG 图中不能被 Chain 在一起的 Operator 会被分隔到不同的 Task 中，一般一个Task占用一个slot，也就是说 Task 是 Flink 中资源调度的最小单位。

在StreamGraph转换为JobGraph过程中，关键在于将多个 StreamNode 优化为一个 JobVertex，对应的 StreamEdge 则转化为 JobEdge，并且 JobVertex 和 JobEdge 之间通过 IntermediateDataSet （中间数据集）形成一个生产者和消费者的连接关系。每个JobVertex就是JobManger的一个任务调度单位（任务Task）。为了避免在这个过程中将关联性很强的几个StreamNode（算子）放到不同JobVertex（Task）中，从而导致因为Task执行产生的效率问题（数据交换（网络传输）、线程上下文切换），Flink会在StreamGraph转换为JobGraph过程中将可以优化的算子合并为一个算子链（也就是形成一个Task）。这样就可以把这条链上的算子放到一个线程中去执行，这样就提高了任务执行效率。

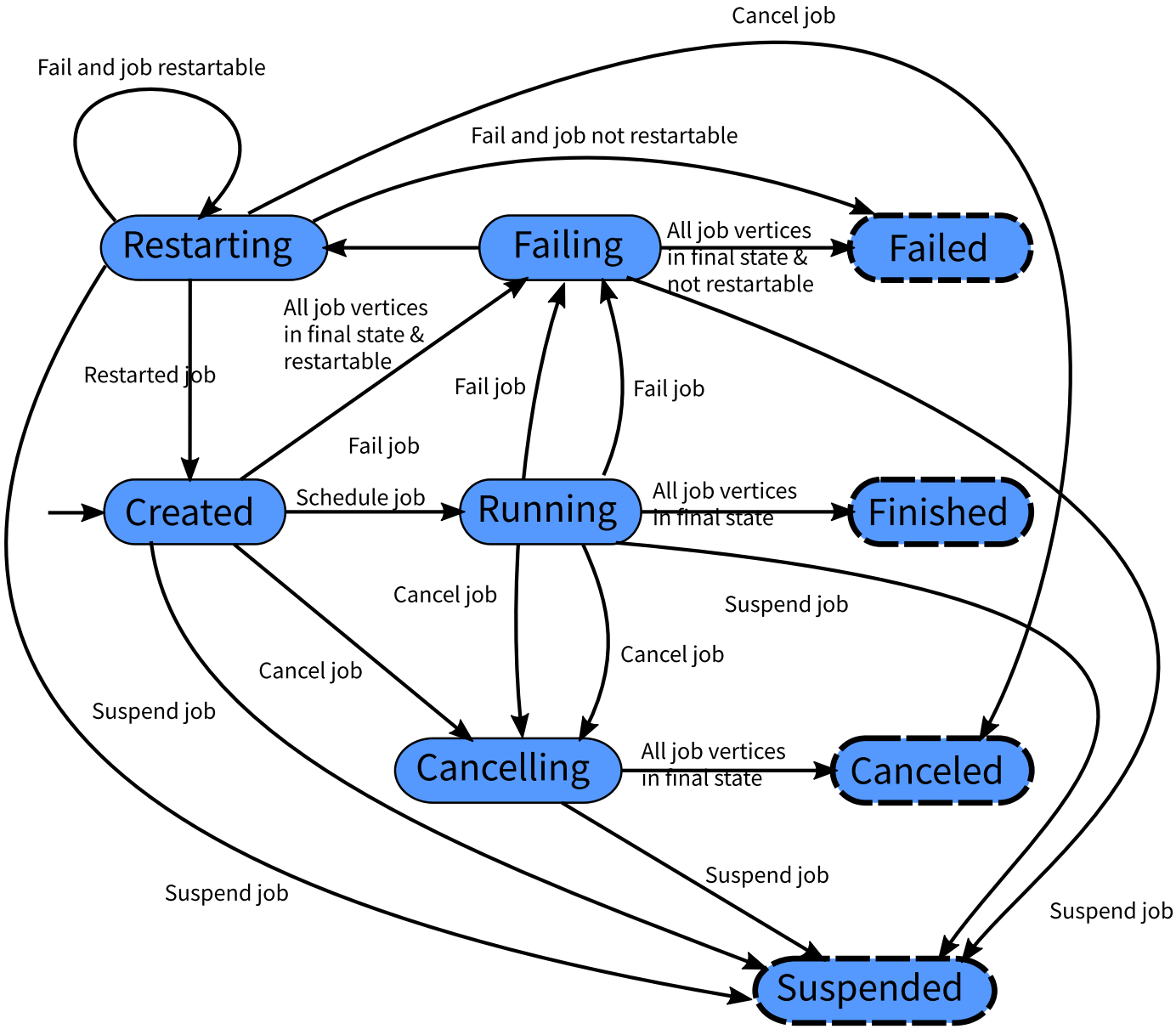
13、Flink中常见的任务状态及如何转换（了解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 实时开发必须了解, 面试很少会被问;** 2. **update 2026.07**

Flink 作业刚开始会处于 created 状态，然后切换到 running 状态，当所有任务都执行完之后会切换到 finished 状态。如果遇到失败的话，作业首先切换到 failing 状态以便取消所有正在运行的 task。如果所有 job 节点都到达最终状态并且 job 无法重启， 那么 job 进入 failed 状态。如果作业可以重启，那么就会进入到 restarting 状态，当作业彻底重启之后会进入到 created 状态。

如果用户取消了 job 话，它会进入到 cancelling 状态，并取消所有正在运行的 task。当所有正在运行的 task 进入到最终状态的时候，job 进入 cancelled 状态。

Finished、canceled 和 failed 会导致全局的终结状态，并且触发作业的清理。跟这些状态不同，suspended 状态只是一个局部的终结。局部的终结意味着作业的执行已经被对应的 JobManager 终结，但是集群中另外的 JobManager 依然可以从高可用存储里获取作业信息并重启。因此一个处于 suspended 状态的作业不会被彻底清理掉。



14、Flink核心的四大基石是什么？（背诵）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 官方没有标准的这个说法,但是是实时计算的核心点,面试问的核心也就是围绕这四大块展开；** 2. **update 2026.07**

Flink的四大基石分别是Checkpoint 、 State 、 Time 、 Window 。
Flink最重要的一个特性，Flink基于**Chandy-Lamport算法(有问过)**实现了一个分布式的一致性的快照，从而提供了一致性的语义。

15、Flink的并行度是什么？（理解背诵）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 理解并且掌握, 初级同学面试问到频率高；** 2. **update 2026.07**

在 Flink 执行过程中，每一个算子（operator）可以包含一个或多个子任务（operator subtask），这些子任务在不同的线程、不同的物理机或不同的容器中完全独立地执行。一个特定算子的子任务（subtask）的个数被称之为其并行度（parallelism）。这样，包含并行子任务的数据流，就是并行数据流，它需要多个分区（stream partition）来分配并行任务。
一般情况下，一个Streaming计算程序的并行度，可以认为就是其所有算子中最大的并行度。一个程序中，不同的算子可能具有不同的并行度。
并行度设置方法，它们的优先级如下：

（1）对于一个算子，首先看在代码中是否单独指定了它的并行度，这个特定的设置优先级最高，会覆盖后面所有的设置。
（2）如果没有单独设置，那么采用当前代码中执行环境全局设置的并行度。
（3）如果代码中完全没有设置，那么采用提交时-p 参数指定的并行度。
（4）如果提交时也未指定-p 参数，那么采用集群配置文件中的默认并行度。

这里需要说明的是，**算子的并行度有时会受到自身具体实现的影响。比如之前我们用到的读取 socket 文本流的算子 socketTextStream，它本身就是非并行的 Source 算子，所以无论怎么设置，它在运行时的并行度都是 1**，对应在数据流图上就只有一个并行子任务。生产实战中怎样设置并行度比较好呢？那就是在代码中只针对算子设置并行度，不设置全局并行度，这样方便我们提交作业时进行动态扩容。

15.1 任务槽和并行度的关系？（理解背诵）

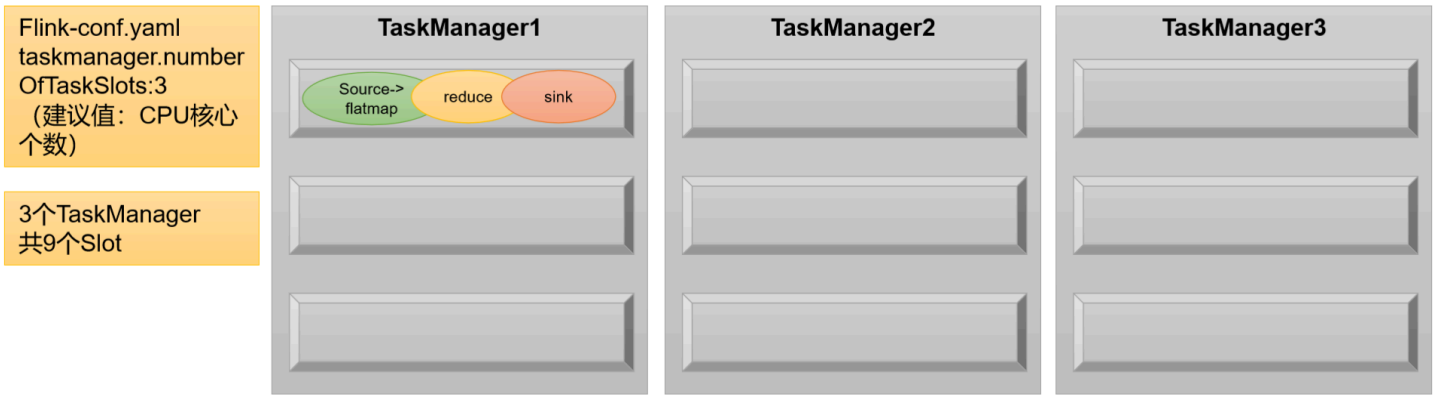
Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 理解并且掌握, 初级同学面试问到频率高；** 2. **update 2026.07**

Slot 和并行度确实都跟程序的并行执行有关，但两者是完全不同的概念。简单来说，**task slot 是静态的概念 ， 是指TaskManager具有的并发执行能力，可以通过参数taskmanager.numberOfTaskSlots 进行配置；而并行度（parallelism）是动态概念，也就是TaskManager 运行程序时实际使用的并发能力，可以通过参数parallelism.default 进行配置。**换句话说，并行度如果小于等于集群中可用 slot 的总数，程序是可以正常执行的，因为 slot 不一定要全部占用，有十分力气可以只用八分；而如果并行度大于可用 slot 总数，导致超出了并行能力上限，那么心有余力不足，程序就只好等待资源管理器分配更多的资源了。因此：TM数量 =算子并行度/slot(向上取整)

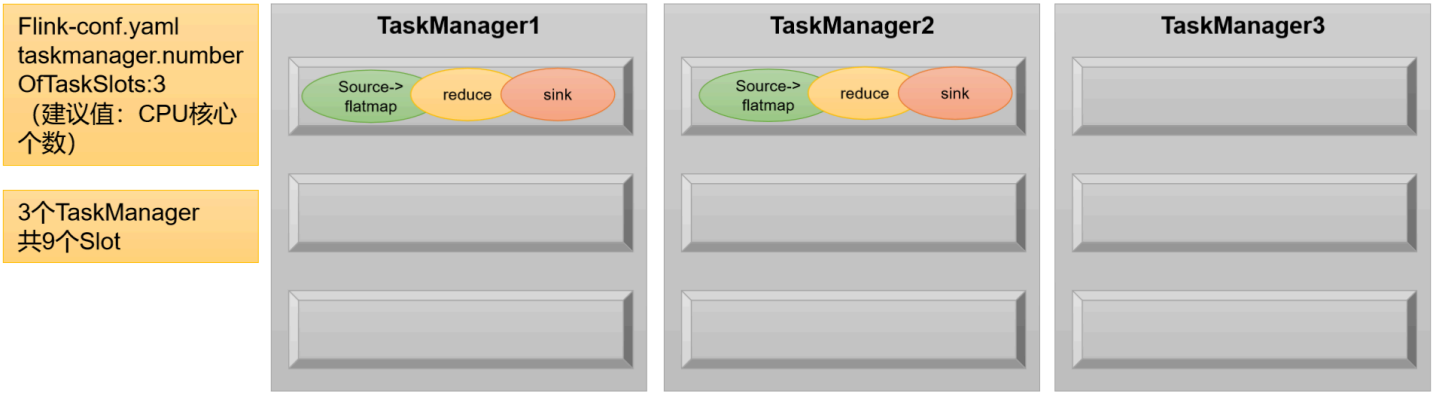
举一个具体的例子。假设一共有 3 个 TaskManager，每一个 TaskManager 中的slot 数量设置为 3 个，那么一共有 9 个 task slot，表示集群最多能并行执行 9个任务。而我们定义 WordCount 程序的处理操作是四个转换算子：source→ flatMap→ reduce→ sink

当所有算子并行度相同时，容易看出 source 和 flatMap 可以合并算子链，于是最终有三个任务节点。

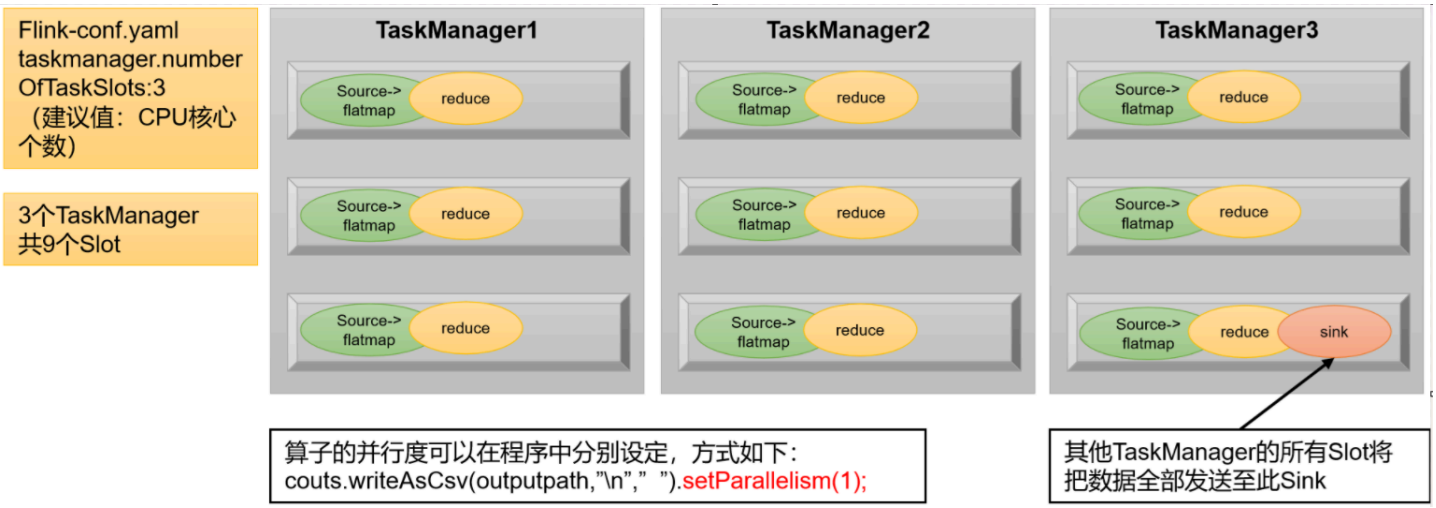
如果我们没有任何并行度设置，而配置文件中默认 parallelism.default=1，那么程序运行的默认并行度为 1，总共有 3 个任务。由于不同算子的任务可以共享任务槽，所以最终占用的 slot只有 1 个。9 个 slot 只用了 1 个，有 8 个空闲，如图中所示。



如果我们更改默认参数，或者提交作业时设置并行度为 2，那么总共有 6 个任务，共享任务槽之后会占用 2 个slot，如图中所示。同样，就有 7 个 slot 空闲，计算资源没有充分利用。



所以可以看到，设置合适的并行度才能提高效率。那对于这个例子，怎样设置并行度效率最高呢？当然是需要把所有的 slot 都利用起来。考虑到 slot 共享，我们可以直接把并行度设置为 9，这样所有 27 个任务就会完全占用 9 个 slot。这是当前集群资源下能执行的最大并行度，计算资源得到了充分的利用，如图中所示。



另外再考虑对于某个算子单独设置并行度的场景。例如，如果我们考虑到输出可能是写入文件，那会希望不要并行写入多个文件，就需要设置 sink 算子的并行度为 1。这时其他的算子并行度依然为 9，所以总共会有 19 个子任务。根据 slot 共享的原则，它们最终还是会占用全部的 9 个 slot，而 sink 任务只在其中一个 slot 上执行，如图 中 Example 4 所示。通过这个例子也可以明确地看到，整个Streaming处理程序的并行度，就应该是所有算子并行度中最大的那个，这代表了运行程序需要的 slot 数量。

15.2、流式计算我们一般希望并行处理、任务之间相互隔离，为什么Flink又允许共享 slot 呢？谈谈你的理解？（理解）

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 理解即可, 面试被问到频率不高;** 2. **update 2026.07**

一个 slot 对应了一组独立的计算资源。在之前不做共享的时候，每个任务都平等地占据了一个 slot，但其实**不同的任务对资源的占用是不同的。**例如这里的前两个任务，source/map 尽管是两个算子合并算子链得到的，但它只是基本的数据读取和简单转换，计算耗时极短，一般也不需要太大的内存空间；而 window 算子所做的窗口操作，往往会涉及大量的数据、状态存储和计算，我们一般把这类任务叫作**“资源密集型”（intensive）任务**。当它们被平等地分配到独立的 slot 上时，实际运行我们就会发现，大量数据到来时 source/map 和 sink任务很快就可以完成，但 window 任务却耗时很久；于是下游的 sink 任务占据的 slot 就会等待闲置，而上游的 source/map 任务受限于下游的处理能力，也会在快速处理完一部分数据后阻塞对应的资源开始等待（相当于处理背压）。这样资源的利用就出现了极大的不平衡，**“忙的忙死，闲的闲死”**。

解决这一问题的思路就是允许 slot 共享。**当我们将资源密集型和非密集型的任务同时放到一个 slot 中，它们就可以自行分配对资源占用的比例，从而保证最重的活平均分配给所有的TaskManager。**slot 共享另一个好处就是允许我们保存完整的作业管道。这样一来，即使某个TaskManager出现故障宕机，其他节点也可以完全不受影响，作业的任务可以继续执行。
另外，同一个任务节点的并行子任务是不能共享 slot 的，所以允许 slot 共享之后，运行作业所需的 slot 数量正好就是作业中所有算子并行度的最大值。这样一来，我们考虑当前集群需要配置多少 slot 资源时，就不需要再去详细计算一个作业总共包含多少个并行子任务了，只看最大的并行度就够了。

当然，**Flink 默认是允许 slot 共享的**，如果希望某个算子对应的任务完全独占一个 slot，或者只有某一部分算子共享 slot，我们也可以通过设置“slot 共享组”（SlotSharingGroup）手动指定：
.map(word -> Tuple2.of(word, 1L)).slotSharingGroup(“1”);

这样，只有属于同一个 slot 共享组的子任务，才会开启 slot 共享；不同组之间的任务是完全隔离的，必须分配到不同的 slot 上。在这种场景下，总共需要的 slot 数量，就是各个 slot共享组最大并行度的总和。

16 . 谈谈Flink 回撤机制的理解

Update	Review	尖叫提醒
健哥	光哥	1. **尖叫提示: 面试中问的频率比较低, 但是对于理解实时很重要, 只需要了解原理,发送的场景, 下游sink支持回撤的要求;** 2. **update 2026.07**

Flink 回撤机制原理：在底层，Flink 的回撤流其实是一种**逻辑上的先删后加**。而是将一次更新拆解为两条带有特殊标识的消息流：

- **撤回消息（Retract）**：标识为 `false`，告诉下游“把之前那条旧数据删掉”。
- **新增消息（Add）**：标识为 `true`，告诉下游“把这条最新的数据插进去”。

举个例子比你在微信里发错了一句话，Flink 的做法不是直接“编辑”那条消息，而是先撤回那条旧消息，紧接着再发一条新的消息。

Flink回撤发生的场景：它的触发往往是因为**流式计算的中间结果发生了变化**。在流计算中，数据是源源不断的，随着新数据的到来或迟到数据的补齐，之前已经计算并发往了下游的聚合计算GROUP BY（如 `COUNT`、`SUM`）或双流JOIN结果可能就不再准确了,或者一些排序TopN, 去重等场景。回撤机制就是 Flink 用来“修正历史计算结果”的橡皮擦。

回撤对下游Sink的要求：根据下游存储系统的特性，将 Sink 分为了几种不同的语义模式，这直接决定了回撤机制能否生效：

Append-only：

- 普通的 Kafka、日志文件、SLS(阿里的日志存储格式)。
- 只能顺序写入新数据，**完全不支持回撤**。如果 Flink 作业上游产生了回撤流（比如做了聚合），强行写入这类 Sink 会直接报错。

Upsert

- **比如**MySQL(有主键)、HBase、Redis、Upsert Kafka。
- 这是生产环境中最常用的模式。这类系统支持主键（Primary Key）。Flink 在写入时，会将回撤流优化为数据库层面的 `UPSERT` 语义（例如 MySQL 的 `INSERT ... ON DUPLICATE KEY UPDATE`）。下游收到数据后，如果发现主键已存在就直接更新，不存在则插入。

其他的olap和数据湖也是支持的，比如：

- **Unique Key： doris, starrocks, clickhouse**
- **Changelog：paimon(表配置的时候设置)**

郑冬综合2502 2026/9/21 21:35:38
涤生大数据版权所有，请勿外传

无标签